MODULE 18 · DOMAIN 3 - APPLICATION DEVELOPMENT

Building agents with MLflow and Custom Agents

The ResponsesAgent interface, configuration, Unity Catalog tools, and tracing every step so you can debug it.

SECTION 3 of 6 EXAM WEIGHT 30% MODULE 18 of 40

admin@sthithapragna.com

Every aircraft carries a flight recorder. **Build agents the same way: every step recorded, every setting outside the code, every tool governed.**

What a Custom Agent is made of

01
Logic
Your agent, in any framework or plain Python.

02
Interface
ResponsesAgent, the standard wrapper.

03
Config
Endpoints, prompts and settings outside the code.

04
Tools
Unity Catalog functions and MCP servers.

05
Traces
MLflow records every step.

ON THE EXAM

The framework inside is your choice. The interface, config, governed tools and traces are what make it a Databricks agent.

Model Serving or Databricks Apps?

MODEL SERVING

A ResponsesAgent class

- Subclass mlflow.pyfunc.ResponsesAgent
- Implement predict and predict_stream
- Log with MLflow, register, deploy
- Legacy for new agents, still examined

DATABRICKS APPS

Decorated functions

- MLflow AgentServer hosts the agent
- Functions marked @invoke and @stream
- Async recommended
- Databricks' recommendation for new agents

THE TIEBREAKER

The exam objectives describe the Model Serving path. New builds are recommended on Apps. The request and response types are the same in both.

The ResponsesAgent pieces

PIECE	WHAT IT IS
`ResponsesAgentRequest`	The input: conversation messages, plus optional custom_inputs
`ResponsesAgentResponse`	The output: a list of output items, such as messages and tool calls
`ResponsesAgentStreamEvent`	One streamed chunk: text deltas, then a done event
`predict()`	Returns a full response. Or @invoke on Apps
`predict_stream()`	Yields stream events. Or @stream on Apps

ON THE EXAM

ResponsesAgent is recommended. ChatAgent and ChatModel are the legacy interfaces, still supported.

Models from code

- The agent lives in a **Python file**, such as agent.py, which ends by calling set_model.
- It is logged with **mlflow.pyfunc.log_model(python_model="agent.py")**.
- MLflow stores the **code itself**, not a pickled object, so it is readable and reviewable.
- Logging also declares **resources** the agent needs, such as endpoints and indexes.
- Then register to Unity Catalog and deploy. Module 23 covers packaging in depth.

ON THE EXAM

Models from code avoids serialisation problems with clients and connections, and keeps the agent reviewable like any other code.

Hard-coded values, or ModelConfig?

HARD-CODED

Settings inside the code

- Endpoint names in the source
- Prompts pasted into functions
- A change means a code edit
- Dev and prod drift apart

MODELCONFIG

Settings in a config file

- mlflow.models.ModelConfig
- development_config points at a YAML file
- Same code, different config
- Logged with the model for reproducibility

THE TIEBREAKER

When the same agent must run with different endpoints, prompts or parameters, put them in ModelConfig, not in the code.

Unity Catalog functions as agent tools

STEP	DETAIL
Define	CREATE FUNCTION in SQL or Python, in a catalog and schema
Describe	A COMMENT on the function and each parameter
Govern	EXECUTE permission decides who, and which agent, can call it
Attach	UCFunctionToolkit, or the managed MCP server for functions
Built in	system.ai.python_exec gives agents a code interpreter

ON THE EXAM

The COMMENT is what the model reads. A vague comment gives a vague tool, exactly as module 04 warned.

What an MLflow trace records

01

AGENT span

The whole request, start to finish.

02

LLM spans

Each model call: prompt and reply.

03

RETRIEVER spans

The query and chunks returned.

04

TOOL spans

Tool name, arguments and result.

05

Timings

How long each span took.

ON THE EXAM

Span types matter. Retrieval judges read RETRIEVER spans. set_retriever_schema tells MLflow which columns hold the text.

Symptom, and where to look in the trace

SYMPTOM	LOOK AT
Wrong tool chosen	TOOL spans, and that tool's description
Same tool called over and over	Repeated TOOL spans, and the stop condition
Slow responses	Span timings, to find the slowest step
Wrong or invented answer	RETRIEVER span contents against the final reply
Crashes	The span carrying the exception

ON THE EXAM

Do not guess from the final answer. Open the trace and find the first span where things went wrong.

Querying the agent

- Use **DatabricksOpenAI**, then **responses.create** with the agent's name as the model.
- Set **stream** to true to receive chunks, which calls the streaming method.
- Pass **custom_inputs** for extra fields, such as a customer ID.
- Ask for **return_trace** in databricks_options to get the trace back.
- Legacy ChatAgent endpoints use the chat completions API instead.

> ON THE EXAM
>
> The Responses API is the standard way to call a ResponsesAgent, whether it is hosted on Apps or Model Serving.

Pennant Outdoor's returns agent, built properly

01

Logic

LangGraph, from module 15.

02

Interface

ResponsesAgent, streaming.

03

Config

Endpoint, prompt and refund limit in YAML.

04

Tools

get_order and create_return as UC functions.

05

Traces

Every tool call recorded and timed.

ON THE EXAM

When finance raised the refund approval limit, only the YAML changed. The code, and its review, did not.

> An order-support agent has become slow. Its MLflow traces show the search_orders tool being called six times per request, with the same arguments each time.
>
> Final answers are correct, but they take far too long.

What should the engineer investigate and fix?

A Move to a larger model with better general reasoning ability

B Add a reranker to the agent's document retrieval step

C The repeated TOOL spans and the agent's stop condition

D Raise the maximum output tokens available to the model

◆ PAUSE AND DECIDE

What should the engineer investigate and fix?

A **Larger model**

Might loop less, but the evidence points at a specific cause.

B **Reranker**

Adds latency, and the slowness is not from retrieval.

C **Repeated TOOL spans and stop condition**

The trace shows the loop. Fix why the agent repeats the call.

D **More output tokens**

Length is not the problem. Repetition is.

THE SIGNAL

The trace already names the symptom: the same tool, same arguments, six times. Start at those spans.

> A team runs one agent in development and production. It needs a different serving endpoint, system prompt and retrieval depth in each.
>
> Today those values are hard-coded, so every change needs a code edit and a new review.

What should the engineer change?

A Move the values into a YAML file read through MLflow ModelConfig

B Keep a separate copy of agent.py for each environment

C Add if statements in the code that check which environment it is in

D Pass the values in as part of each user message at request time

◆ PAUSE AND DECIDE

What should the engineer change?

A ModelConfig YAML

Same code, different config per environment, logged with the model.

B A copy per environment

Two codebases that will drift apart.

C Environment if statements

Still hard-coded, just in more places.

D Values in user messages

Users could change the prompt and endpoint. Unsafe.

THE SIGNAL

Same agent, different values per environment, no code edits. That is exactly what ModelConfig is for.

> An engineer runs RetrievalGroundedness on an agent's traces, but gets no retrieval assessments.
>
> The agent retrieves chunks through a custom Python function that is not traced as a retriever.

What is the fix?

A Switch to the Correctness judge, which needs no traces at all

B Use a larger judge model so it can find the retrieved context

C Add more documents to the index so retrieval returns something

D Trace the retriever with @mlflow.trace(span_type="RETRIEVER")

◆ PAUSE AND DECIDE

What is the fix?

A Switch to Correctness

Needs expected facts, and does not assess grounding.

B Larger judge model

No model can read a span that was never recorded.

C More documents

Retrieval already works. It just is not traced.

D Trace it as a RETRIEVER span

Retrieval judges read RETRIEVER spans. Now there is one.

THE SIGNAL

Not traced as a retriever. The retrieval judges need a RETRIEVER span; give them one.

> An agent needs to look up a customer's loyalty tier with a SQL query. Security wants the lookup governed by permissions, discoverable, and reusable by other agents.
>
> The team is deciding how to provide the tool.

Which approach meets the requirement?

A. A Python function defined inside the agent's own code

B. A Unity Catalog function with a COMMENT, attached as a tool

C. The SQL query pasted into the system prompt for the model

D. A personal access token letting the agent run any SQL query

◆ PAUSE AND DECIDE

Which approach meets the requirement?

A Function in the agent code

Works, but not governed, discoverable or reusable.

B Unity Catalog function

Governed by EXECUTE, discoverable, reusable by any agent.

C SQL in the system prompt

The model cannot run SQL by reading it.

D Token for any SQL

The opposite of least privilege.

THE SIGNAL

Governed, discoverable and reusable. Those three words describe a Unity Catalog function tool.

What to carry forward

- **ResponsesAgent**: predict and predict_stream on Model Serving; **@invoke and @stream** on Apps.
- Log **models from code**, with resources declared. ChatAgent is legacy.
- Settings in **ModelConfig**, not in the code.
- Tools as **Unity Catalog functions**, described by COMMENT, governed by EXECUTE.
- Debug from the trace: **find the first span** where things went wrong.

> ON THE EXAM
>
> Next: module 19 brings Genie Agents into multi-agent systems, for questions over tables.